

Préparation de l'oral — Stratégie de l'IA Hnefatafl

1. Présentation générale

Notre IA combine deux éléments :

1. Un algorithme **minimax avec élagage alpha-bêta** explore les conséquences possibles des coups.
2. Une **fonction heuristique** estime la qualité des positions lorsque la recherche ne peut pas atteindre la fin de la partie.

Hnefatafl étant asymétrique, les deux camps ont des stratégies différentes :

- **Rouge**, l'attaquant, doit capturer le roi.
- **Noir**, le défenseur, doit faire atteindre un coin au roi.

Le fonctionnement général est :

Serveur

- Client reçoit la position
- CPUPlayer génère les coups légaux
- Minimax simule les réponses adverses
- HeuristicEvaluator évalue les feuilles
- Le meilleur coup est envoyé au serveur

2. Le rôle de minimax

Minimax est adapté aux jeux adversariaux :

- notre IA est le joueur **MAX** et cherche le score le plus élevé ;
- l'adversaire est le joueur **MIN** et cherche le score le plus faible pour nous.

Exemple :

Coup A

- └ Réponse adverse 1 → 500
 - └ Réponse adverse 2 → -200
- MIN choisira -200

Coup B

- └ Réponse adverse 1 → 100
 - └ Réponse adverse 2 → 50
- MIN choisira 50

MAX choisit B, car $50 > -200$.

Même si le coup A peut mener à une bonne position, l'IA suppose que l'adversaire jouera sa meilleure réponse. Elle choisit donc le coup dont la **pire conséquence possible** reste la meilleure.

Après avoir simulé un de nos coups à la racine, la recherche continue par un nœud MIN, puisque c'est alors à l'adversaire de jouer.

3. L'élagage alpha-bêta

Un arbre minimax devient rapidement très grand. L'élagage alpha-bêta permet d'ignorer une branche lorsqu'on sait qu'elle ne pourra plus changer la décision finale.

- **Alpha** représente le meilleur score déjà garanti pour MAX.
- **Bêta** représente le meilleur score déjà garanti pour MIN.
- Lorsque `beta <= alpha`, le reste de la branche peut être coupé.

Exemple :

MAX possède déjà un coup qui garantit 100.

Dans une autre branche, MIN trouve une réponse qui donne 50.

MAX ne choisira pas cette branche, car MIN peut déjà le limiter à 50.

Le reste de cette branche est donc inutile.

Si toute une profondeur est terminée, alpha-bêta donne **le même résultat que minimax**. Il réduit uniquement le nombre de nœuds explorés.

4. L'ordre des coups

L'efficacité d'alpha-bêta dépend de l'ordre dans lequel les coups sont visités.

Le moteur essaie en priorité :

1. le meilleur coup trouvé à la profondeur précédente ;
2. les coups qui semblent effectuer une capture ;
3. les autres coups.

Trouver rapidement de bons coups améliore les bornes alpha et bêta, ce qui permet davantage de coupures.

La fonction `isLikelyCapture` est une estimation rapide. Elle ne supprime aucun coup et ne change donc pas le résultat théorique d'une profondeur terminée. Elle change uniquement l'ordre d'exploration.

5. L'approfondissement itératif

Le serveur impose une limite de **5 secondes par coup**. Le client réserve 500 ms pour le réseau et la JVM, ce qui donne environ **4,5 secondes de recherche**.

Le moteur ne choisit pas une profondeur fixe. Il recherche successivement :

```
profondeur 1
profondeur 2
profondeur 3
profondeur 4
...
jusqu'à l'expiration du temps
```

À chaque itération :

- tous les coups sont réévalués ;
- le meilleur coup précédent est essayé en premier ;
- le résultat est conservé uniquement si la profondeur est entièrement terminée.

Si le temps expire pendant la profondeur 5, le moteur utilise le résultat complet de la profondeur 4. Cela évite de jouer un résultat provenant d'un arbre partiellement exploré.

Cette méthode s'adapte à la complexité de la position :

- une position simple permet d'aller plus profondément ;
- une position complexe respecte quand même la limite de temps.

L'horloge est vérifiée à la racine et environ tous les 256 nœuds. Cela limite le coût des appels répétés à `System.nanoTime()`.

6. Les feuilles et les fins de partie

Avant de poursuivre une branche, le moteur vérifie :

1. si le roi s'est échappé ;
2. si le roi a été capturé ;
3. si la profondeur maximale est atteinte ;
4. si le joueur actif possède un coup légal.

Les résultats terminaux utilisent :

```
WIN_SCORE = 1 000 000
```

Du point de vue du camp contrôlé :

- victoire : `+1 000 000` ;
- défaite : `-1 000 000` ;
- aucun coup légal : `0`, donc match nul.

Le score terminal est beaucoup plus élevé que les bonus heuristiques. Une vraie victoire reste donc toujours préférable à un simple avantage positionnel.

7. Pourquoi une heuristique est nécessaire

L'arbre complet de Hnefatafl est trop grand pour être exploré jusqu'à la fin en 4,5 secondes.

Lorsque minimax atteint sa profondeur maximale, l'heuristique répond à la question :

Si la recherche s'arrête maintenant, cette position semble-t-elle favorable ou défavorable à notre camp ?

Ce n'est pas une preuve de victoire. C'est une estimation fondée sur :

- le matériel restant ;
- les possibilités d'évasion du roi ;
- son niveau d'encercllement ;
- les gardes noirs autour du roi ;
- les pièces rouges qui risquent d'être capturées.

8. L'asymétrie de l'évaluation

La partie positionnelle est d'abord exprimée du point de vue du défenseur :

```
defenderPositionScore
```

Un score positionnel défenseur élevé correspond à :

- un roi mobile ;
- des couloirs ouverts ;
- des coins accessibles ;
- des gardes autour du roi.

Un score faible correspond à :

- un roi encerclé ;
- des couloirs bloqués ;
- des rouges bien placés ;
- des gardes menacées.

Les formules finales sont :

```
Score noir =  
    matériel défenseur  
+ position défenseur
```

```
Score rouge =  
    matériel attaquant
```

- position défenseur
- pénalité des rouges menacés

Ce qui aide le roi augmente donc le score noir, mais diminue le score rouge.

9. L'évaluation du matériel

9.1 Pour le défenseur

```
matériel défenseur =  
    nombre de noirs × 2 000  
    - nombre de rouges × 1 000
```

Au départ :

```
12 noirs × 2 000 = 24 000  
24 rouges × 1 000 = 24 000
```

Le matériel défenseur initial est donc approximativement neutre.

Capter un rouge améliore le score défenseur de 1 000. Perdre un noir lui coûte 2 000, car les noirs sont moins nombreux.

9.2 Pour l'attaquant

```
matériel attaquant =  
    nombre de rouges × 3 000  
    - nombre de noirs × 1 000
```

Perdre un rouge est très pénalisant, car plusieurs attaquants sont nécessaires pour :

- bloquer les couloirs ;
- éliminer les gardes ;
- fermer les quatre côtés du roi.

Cette pondération évite que les rouges sacrifient leurs pièces pour un petit avantage positionnel temporaire.

L'important n'est pas le score absolu, mais la différence de score entre les positions comparées.

10. Le raycasting

Le roi se déplace comme une tour. L'évaluation lance donc quatre rayons :

haut, bas, gauche, droite

Chaque rayon avance jusqu'à rencontrer :

- une pièce ;
- une case non traversable ;
- la limite du plateau.

Les cases **EMPTY** et **SPECIAL** sont traversables par le roi.

Pour chaque direction, le moteur mesure :

- la distance maximale atteignable ;
- si le roi peut atteindre directement un coin ;
- s'il peut atteindre un bord qui mène ensuite vers un coin.

Les principaux poids sont :

```
Coin accessible en un coup      → +50 000 pour le défenseur
Bord menant potentiellement au coin → +8 000
Chaque case accessible          → +40
Chaque coin directement visible → +3 000
```

Le bonus de 50 000 est élevé parce qu'un couloir direct vers un coin représente une menace de victoire immédiate.

Le raycasting est plus pertinent qu'une simple distance au coin. Un roi peut être proche géométriquement d'un coin, mais incapable de l'atteindre à cause d'une pièce qui bloque le chemin.

Sa limite est qu'il analyse le plateau actuel. C'est minimax qui examine ensuite les réponses de l'adversaire.

11. L'encerclement du roi

L'évaluateur inspecte les quatre cases orthogonales autour du roi.

Une direction est hostile si elle correspond à :

- une pièce rouge ;
- une case spéciale hostile selon les règles ;
- l'extérieur du plateau.

Chaque côté hostile retire 300 au score défenseur :

```
côté hostile      → -300
au moins 2 hostiles → -1 000 supplémentaires
au moins 3 hostiles → -4 000 supplémentaires
```

Ces pénalités rendent la position moins bonne pour le défenseur et, par inversion, meilleure pour le rouge.

12. Les côtés ouverts et le blocage des lignes

Une case vide ou une pièce noire adjacente au roi compte comme un côté ouvert :

côté ouvert → +400 pour le défenseur

Une case vide permet au roi de se déplacer. Une pièce noire peut le protéger et compliquer la fermeture du filet.

L'évaluateur examine également chaque ligne et colonne partant du roi. Si le premier obstacle visible est rouge, ce rouge bloque un couloir :

rouge bloquant une ligne du roi → -120 pour le défenseur

Cela pousse l'attaque à bloquer les véritables trajectoires du roi, plutôt qu'à simplement rapprocher ses pièces.

13. Les gardes noirs

Une pièce noire directement adjacente au roi est considérée comme une garde :

garde noire → +1 800 pour le défenseur

Le rouge doit généralement éliminer cette garde avant de pouvoir fermer la case qu'elle occupe.

L'évaluateur vérifie aussi si une garde peut être capturée au prochain coup :

1. une pièce rouge ou une case spéciale sert d'enclume ;
2. la case opposée est vide ;
3. un autre rouge peut glisser jusqu'à cette case.

Une garde menacée donne :

garde capturable → -3 500 pour le défenseur

Cette vérification ajoute une petite vision tactique d'un coup à l'évaluation statique.

14. La forteresse

Une situation importante a été observée pendant les tests :

```
trois côtés hostiles autour du roi  
+  
une garde noire sur le quatrième côté
```

Le roi semble presque capturé. Pourtant, si rouge capture immédiatement la garde :

1. la garde disparaît ;
2. sa case devient vide ;
3. le roi peut se déplacer sur cette case ;
4. le filet rouge est brisé.

Cette position est donc une **forteresse défensive**, et non une capture garantie.

Lorsque cette situation est reconnue, l'évaluation ajoute :

```
+12 000 par garde au défenseur
```

Pour rouge, cela devient une forte pénalité. Il est incité à :

1. bloquer les autres sorties ;
2. retirer les gardes au bon moment ;
3. fermer ensuite l'encerclement final.

Cette règle distingue un véritable filet d'une position qui semble agressive, mais qui donne en réalité une sortie au roi.

15. Les rouges menacés

Le matériel voit seulement les pièces présentes. À l'horizon de recherche, une pièce rouge peut encore être sur le plateau tout en étant capturable au prochain coup.

Pour chaque rouge, l'évaluateur vérifie :

1. si une pièce noire, le roi ou une case spéciale forme une enclume ;
2. si la case opposée est vide ;
3. si un noir ou le roi peut glisser jusqu'à cette case.

Chaque rouge menacé reçoit :

```
-2 000 pour l'attaquant
```

Si la capture est réellement jouée dans l'arbre, la disparition de la pièce coûtera ensuite 3 000 points de matériel. Cette pénalité anticipée réduit les sacrifices inutiles à la limite de l'horizon.

16. L'anti-répétition

Le client mémorise les positions déjà rencontrées dans `seenPositions`.

À la racine :

```
position déjà vue → -200 000
```

Le moteur mémorise également le dernier coup joué :

```
A → B suivi immédiatement de B → A → -25 000
```

Ces pénalités empêchent les longues boucles et les allers-retours.

Elles sont appliquées à la racine, car elles servent à influencer le coup réellement joué sans rendre toute l'évaluation interne dépendante de l'historique.

Ce mécanisme n'est pas une implémentation exacte de la règle des trois répétitions : une position déjà vue est pénalisée dès sa répétition. C'est une solution pratique contre les boucles.

La pénalité reste inférieure à `WIN_SCORE`, donc une vraie victoire reste prioritaire.

17. Exemple d'une décision rouge

Supposons que rouge hésite entre deux coups.

Coup A : avancer près du roi

- ajoute un côté hostile : positif ;
- rend la pièce rouge capturable : négatif ;
- ouvre un rayon du roi vers un coin : très négatif ;
- répète peut-être une position : négatif.

Coup B : bloquer une ligne du roi

- ferme un rayon d'évasion ;
- réduit la mobilité du roi ;
- place un rouge sur une ligne importante ;
- conserve le matériel ;
- ne répète pas la position.

Même si A semble visuellement plus agressif, B peut être meilleur. La stratégie rouge consiste à construire un **filet stable**, pas simplement à rapprocher ses pièces du roi.

Minimax vérifie ensuite si noir dispose d'une réponse qui réfute B. Le coup retenu est celui dont la pire réponse adverse reste la plus favorable.

18. Forces de la stratégie

- Respect de la limite de temps grâce à l'approfondissement itératif.
 - Réduction du nombre de nœuds grâce à alpha-bêta.
 - Raycasting cohérent avec le déplacement réel du roi.
 - Traitement distinct des objectifs rouge et noir.
 - Conservation du matériel rouge nécessaire à la capture finale.
 - Analyse tactique des gardes et des pièces en prise.
 - Gestion des forteresses autour du roi.
 - Réduction des répétitions et des allers-retours.
 - Priorité absolue donnée aux victoires réelles.
-

19. Limites et améliorations possibles

Poids manuels

Les poids de l'heuristique ont été ajustés expérimentalement. Ils ne sont pas appris automatiquement.

Profondeur limitée

L'IA peut manquer une combinaison tactique plus profonde que l'horizon atteint en 4,5 secondes.

Absence de table de transposition

Une position atteinte par plusieurs séquences peut être recalculée plusieurs fois.

Absence de quiescence complète

La recherche peut s'arrêter au milieu d'une séquence tactique ou de captures.

Raycasting statique

Le raycasting étudie la position actuelle. Il dépend de minimax pour examiner la réponse de l'adversaire.

Répétition approximative

Le moteur pénalise toute position déjà vue au lieu de compter exactement trois répétitions.

Ordre des captures approximatif

`isLikelyCapture` sert seulement à ordonner les coups. Les véritables captures restent déterminées par `Board.play`.

Validation réseau incomplète

`Board.play` suppose qu'un coup reçu est valide et le message 4 du serveur produit encore une exception.

Ces limites représentent des compromis entre la qualité de jeu, la complexité du programme et le temps de calcul disponible.

20. Présentation orale courte

Notre stratégie repose sur un minimax avec élagage alpha-bêta. Notre joueur maximise son score et suppose que l'adversaire choisira toujours la réponse qui le minimise. Comme le serveur impose cinq secondes, nous utilisons un approfondissement itératif avec environ 4,5 secondes de recherche et nous conservons le dernier résultat entièrement terminé.

La fonction d'évaluation est asymétrique. Pour les noirs, elle récompense la conservation des pièces, la mobilité du roi et les couloirs ouverts vers les coins. Ces couloirs sont mesurés par raycasting dans les quatre directions, selon le déplacement en tour du roi. Pour les rouges, on cherche au contraire à fermer ces rayons, entourer le roi, éliminer ses gardes et conserver assez d'attaquants pour terminer la capture.

Nous avons également ajouté une détection des pièces rouges menacées, une gestion particulière des forteresses où le roi est protégé par une garde, ainsi que des pénalités contre les répétitions et les allers-retours. Alpha-bêta et l'ordre des coups permettent d'atteindre une meilleure profondeur sans dépasser le temps.

21. Formules à mémoriser

```
Recherche :  
meilleur coup = meilleur résultat contre la meilleure réponse adverse  
  
Noir :  
matériel + mobilité du roi + chemins vers les coins - encerclement  
  
Rouge :  
matériel - possibilités d'évasion du roi - rouges menacés  
  
Objectif rouge :  
construire un filet stable  
  
Objectif noir :  
conserver ou ouvrir un couloir vers un coin
```

22. Questions probables

Pourquoi ne pas utiliser uniquement la distance entre le roi et un coin ?

Parce que la distance ne considère pas les obstacles. Le raycasting vérifie les couloirs réellement traversables.

Pourquoi utiliser des scores différents pour le matériel rouge et noir ?

Les camps ne possèdent pas le même nombre de pièces et n'ont pas le même objectif. Le rouge a besoin de plusieurs pièces coordonnées pour terminer l'encerclement.

Alpha-bêta change-t-il la décision de minimax ?

Non, si la profondeur est entièrement terminée. Il évite seulement d'explorer des branches qui ne peuvent plus modifier le résultat.

Pourquoi utiliser l'approfondissement itératif ?

Il garantit un coup provenant d'une recherche complète tout en utilisant presque tout le temps disponible.

Pourquoi une position avec trois rouges et une garde peut-elle être bonne pour noir ?

Parce que capturer la garde libère sa case. Le roi peut alors s'y déplacer avant que rouge ne ferme le quatrième côté.

Pourquoi pénaliser les rouges menacés s'ils ne sont pas encore capturés ?

Pour éviter l'effet d'horizon : une pièce peut être encore présente à la dernière profondeur alors que sa capture est immédiate.

Pourquoi les pénalités de répétition sont-elles appliquées seulement à la racine ?

Elles servent à choisir le prochain coup réel sans rendre toutes les positions simulées dépendantes de l'historique complet de la partie.